

Design and RTL Simulation of a Custom 4-Bit Harvard Microprocessor

1. Overview:

This document details the Register-Transfer Level (RTL) design and timing simulation of a custom 4-bit soft-core microprocessor. The system was engineered from the ground up using Verilog hardware description language. The processor features a strict Harvard architecture, separating instruction memory from data storage, and is built around a fully functional 74181-style Arithmetic Logic Unit (ALU).

2. Architectural specifications:

- Architecture type: Harvard (that is separate instruction and data pathways)
- Datapath width: 4-bit (ALU, registers, internal data buses)
- Instruction width: 16-bit
- Program counter (PC): 8-bit synchronous counter (supporting 256 words of instruction memory)
- Execution model: Single-cycle execution with purely combinational FSM control.

3. Instruction set architecture (ISA) and optimization

The command line or instructions are 256 words deep, with each word having 16-bits each. These 16-bits command line is broken down in segments by the Control Unit FSM and required input signals are given to ALU, registers and Program Counter. The top 4-bits of the command are used to determine state of the FSM.

State A: The ALU State (Top 4-bits = 0000)

If the top 4 bits are zero, the FSM enables the Register File (Write Enable = 1) and routes the remaining 12 bits to the ALU and Registers to perform arithmetic or logic operations.

- [11:8]: ALU Select Bits (e.g., 1001 for ADD)
- [7]: ALU Mode (0 = Arithmetic, 1 = Logic)
- [6]: Carry-In (Active Low)
- [5:4]: Destination Register Address
- [3:2]: Read A Register Address
- [1:0]: Read B Register Address

State B: The JUMP State (Top 4-bits $\neq$ 0000)

If the top 4 bits contain a non-zero value, the FSM instantly switches to branching mode. It disables register writing to prevent corrupting data with "Don't Care" ALU outputs, treats those exact same top 4 bits as an absolute memory address, and forces the Program Counter to jump to that location on the next clock edge.

4. Hardware Components

The microprocessor is modularized into the following structural components:

Arithmetic and Logic Unit (ALU):

The ALU module designed in this project is inspired from the classic IC-74181, supporting 16 arithmetic and 16 logic operations with two 4-bits input A, B and a 4-bits output F. The 4-bits Select line bus (S) decides the operation and Mode of operation signal (M) decides whether to have a logic operation or arithmetic.

Register file (4X4):

A 4x4-bit SRAM block is built from synchronous D-Flip-Flops. It features dual asynchronous read ports (multiplexed) and a single synchronous write port controlled via a 2:4 decoder. The decoder decides in which register to store data in whereas the multiplexer chooses which register's stored value to be read.

Program Counter (PC):

An 8-bit synchronous counter with asynchronous reset and parallel load capabilities to jump from current state of the counter to another as instructed in the command line, that is for JUMP state. The counter output also activates the corresponding 256 rows of commands/words in ROM.

Instruction Memory (ROM):

An $8 \times (2^8)$ ROM, with 8 address lines, and 256 words of 16-bits each. Each word of 16-bit is a command operated in 1 clock cycle by the FSM.

Control Unit (FSM):

A combinational logic block to decode the hybrid ISA and route control signals to the data path. The control signals are Write Enable, Jump Enable, and from the slicing the command line from ROM: Reading register signals A, B, ALU select line S, mode M, carry in Cn and 8-bit JUMP code.

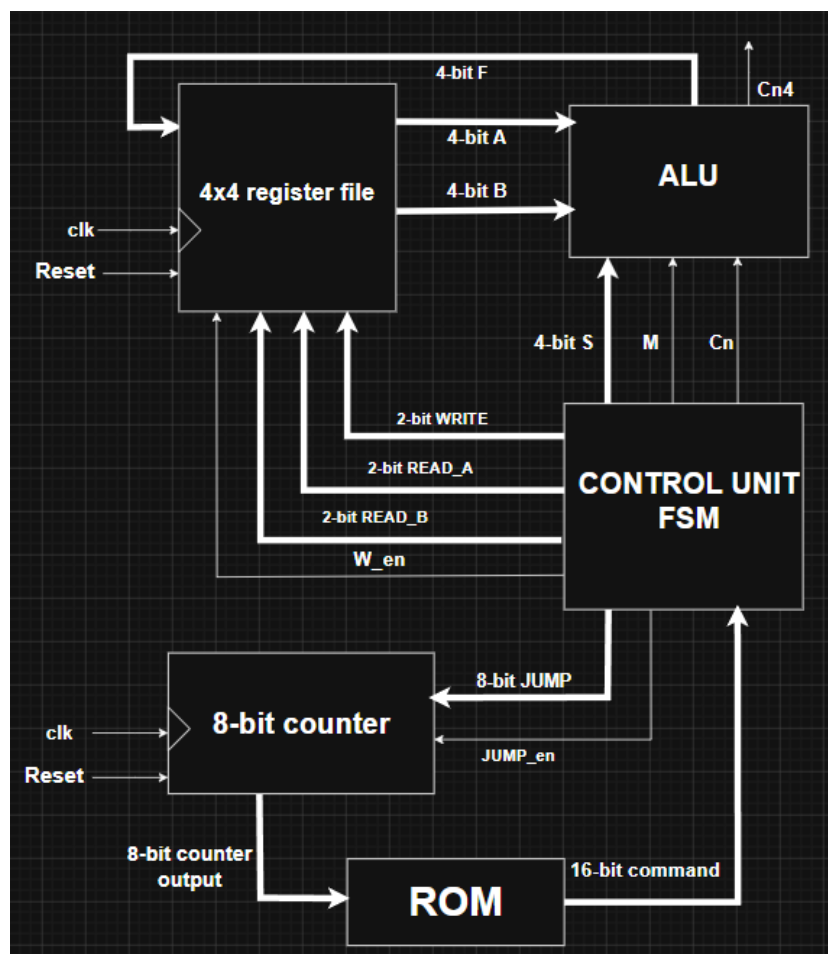


Figure 1: circuit/block diagram of the 4-bit CPU

Selection				Active-low data			Active-high data		
				Logic M = H	Arithmetic M = L		Logic M = H	Arithmetic M = L	
S3	S2	S1	S0		Cn = L (no carry)	Cn = H (carry)		Cn = H (no carry)	Cn = L (carry)
L	L	L	L	$\overline{A}$	A minus 1	A	$\overline{A}$	A	A plus 1
L	L	L	H	$\overline{AB}$	AB minus 1	AB	$\overline{A+B}$	$A+B$	$(A+B)$ plus 1
L	L	H	L	$\overline{A+B}$	$\overline{AB}$ minus 1	$\overline{AB}$	$\overline{AB}$	$A+\overline{B}$	$(A+\overline{B})$ plus 1
L	L	H	H	Logical 1	-1 (two's complement)	0 (zero)	Logical 0	-1 (two's complement)	0 (zero)
L	H	L	L	$\overline{A+B}$	A plus $(A+\overline{B})$	A plus $(A+\overline{B})$ plus 1	$\overline{AB}$	A plus $\overline{AB}$	A plus $(\overline{AB})$ plus 1
L	H	L	H	$\overline{B}$	AB plus $(A+\overline{B})$	AB plus $(A+\overline{B})$ plus 1	$\overline{B}$	$(A+B)$ plus $\overline{AB}$	$(A+B)$ plus $\overline{AB}$ plus 1
L	H	H	L	$\overline{A \oplus B}$	A minus B minus 1	A minus B	$A \oplus B$	A minus B minus 1	A minus B
L	H	H	H	$A+\overline{B}$	$A+\overline{B}$	$A+\overline{B}$ plus 1	$\overline{AB}$	$\overline{AB}$ minus 1	$\overline{AB}$
H	L	L	L	$\overline{AB}$	A plus $(A+B)$	A plus $(A+B)$ plus 1	$\overline{A+B}$	A plus AB	A plus AB plus 1
H	L	L	H	$A \oplus B$	A plus B	A plus B plus 1	$\overline{A \oplus B}$	A plus B	A plus B plus 1
H	L	H	L	B	$\overline{AB}$ plus $(A+B)$	$\overline{AB}$ plus $(A+B)$ plus 1	B	$(A+\overline{B})$ plus AB	$(A+\overline{B})$ plus AB plus 1
H	L	H	H	$A+B$	$A+B$	$(A+B)$ plus 1	AB	AB minus 1	AB
H	H	L	L	Logical 0	A plus A	A plus A plus 1	Logical 1	A plus A	A plus A plus 1
H	H	L	H	$\overline{AB}$	AB plus A	AB plus A plus 1	$A+\overline{B}$	$(A+B)$ plus A	$(A+B)$ plus A plus 1
H	H	H	L	AB	$\overline{AB}$ plus A	$\overline{AB}$ plus A plus 1	$A+B$	$(A+\overline{B})$ plus A	$(A+\overline{B})$ plus A plus 1
H	H	H	H	A	A	A plus 1	A	A minus 1	A

Figure 2: Operation table for ALU IC-74181

5. Code snippets

Module for Control Unit FSM:

```
//control unit FSM module

module FSM (
    input [15:0]commands,    //16 bit command line
    output jump_en,          //jump enable
    output [7:0]jump,        //8 bit jump code
    output [3:0]S,           //4 bit Select line for ALU
    output M,                //Logic or arithmetic operation mode
    output Cn,               //carry in
    output [1:0]W,           //2 bit WRITE signal to register
    output [1:0]Read_A,      //2 bit READ signal to register
    output [1:0]Read_B,      //2 bit READ signal to register
    output w_en              //write enable
);
    //slicing the 16-bit command line in desired variables
    assign S = commands[11:8];
    assign M = commands[7];
    assign Cn = commands[6];
    assign W = commands[5:4];
    assign Read_A = commands[3:2];
    assign Read_B = commands[1:0];

    //8bit jump address
    assign jump = {4'b0000,commands[15:12]};
    //if jump != 00000000 jump_en HIGH
    assign jump_en = jump[3] | jump[2] | jump[1] | jump[0];

    //when operation is not to jump we save the output
    assign w_en = ~jump_en;

endmodule
```

Figure 3: Code snippet for Control Unit FSM module

Module for top-level CPU:

Below is a subset of the top-level structural Verilog. This demonstrates the closed-loop data path integration, specifically highlighting how the internal buses route the instruction memory into the Control Unit, and how the resulting control signals are distributed to the Register File and ALU.

```

//4-bit cpu module
module cpu (
    input clk,        //clock
    input reset       //reset
);

    wire [7:0]count;           //counter to rom
    wire [15:0]commands;       //rom to fsm
    wire jump_en; wire [7:0]jump; //fsm to counter
    wire [3:0]S; wire M, Cn;    //fsm to alu
    wire [1:0]W, Read_A, Read_B; wire w_en; //fsm to register
    wire [3:0]A, B;            //register to alu
    wire [3:0]F;               //alu to register
    wire Cn4, eq;

    //instantiation of 8 bit counter
    counter PC (.clk(clk), .Q(count), .reset(reset), .jump_en(jump_en), .jump(jump));
    //instantiation of ROM
    rom Instruction_memory (.address_lines(count), .commands(commands));
    //instantiation of FSM unit
    FSM_Control_unit (.commands(commands), .jump_en(jump_en), .jump(jump), .S(S),
        .M(M), .Cn(Cn), .W(W), .Read_A(Read_A), .Read_B(Read_B), .w_en(w_en));
    //instantiation of 4X4 register file
    register_block Storage (.I(F), .W(W), .Read_A(Read_A), .Read_B(Read_B),
        .reset(reset), .w_en(w_en), .clk(clk), .A(A), .B(B));
    //instantiation of IC-74181 inspired ALU
    ALU_logic_unit (.A(A), .B(B), .S(S), .M(M), .Cn(Cn), .F(F), .Cn4(Cn4), .eq(eq));

endmodule

```

Figure 4: Code snippet for top-level CPU module

Module for Memory instructions (ROM):

```

//ROM module
module rom (
    input [7:0] address_lines, //8-bit address lines or 8 address lines
    output reg [15:0] commands //16 bit command line
);
    always @(*) begin
        case (address_lines)
            // [0000]-[0000]-[0]-[0]-[00]-[00]-[00]
            // [jump]-[SSSS]-[M]-[c]-[WR]-[RA]-[RB]
            |
            //when counter = 0 | F = A + 1
            8'd0 : commands = 16'b0000_0000_0_0_00_00_00;
            //when counter = 1 | F = A + B + 1
            8'd1 : commands = 16'b0000_1001_0_0_10_00_01;
            //when counter = 2 | F = A - B
            8'd2 : commands = 16'b0000_0110_0_0_11_10_01;
            //when counter = 3 | F = A + B + 1
            8'd3 : commands = 16'b0000_1001_0_0_11_00_10;
            //when counter = 4 | F = A + B
            8'd4 : commands = 16'b0000_1001_0_1_01_11_10;
            //when counter = 5 | F = A ~^ B
            8'd5 : commands = 16'b0000_1001_1_0_00_01_11;
            //when counter = 6 | F = A - 1
            8'd6 : commands = 16'b0000_1111_0_1_11_00_11;
            //when counter = 7 | JUMP
            8'd7 : commands = 16'b0101_0000_0_0_00_00_00;
            //we can write upto a total of 255 command lines of 16 bits each
            default : commands = 16'b0000_0011_1_0_00_01_10;
        endcase
    end
endmodule

```

Figure 5: Code snippet for ROM module

6. Simulation and Verification:

The architecture was compiled and verified using Icarus Verilog, with timing analysis performed via GTKWave. A multi-line test program was loaded into the ROM, as shown above in the code snippet ROM module, to verify instruction fetching, ALU arithmetic correctness, register write-back, and zero-flag jump loops.

The waveform analysis below demonstrates the successful execution of the datapath, including the expected one-cycle delay inherent to synchronous branching logic.

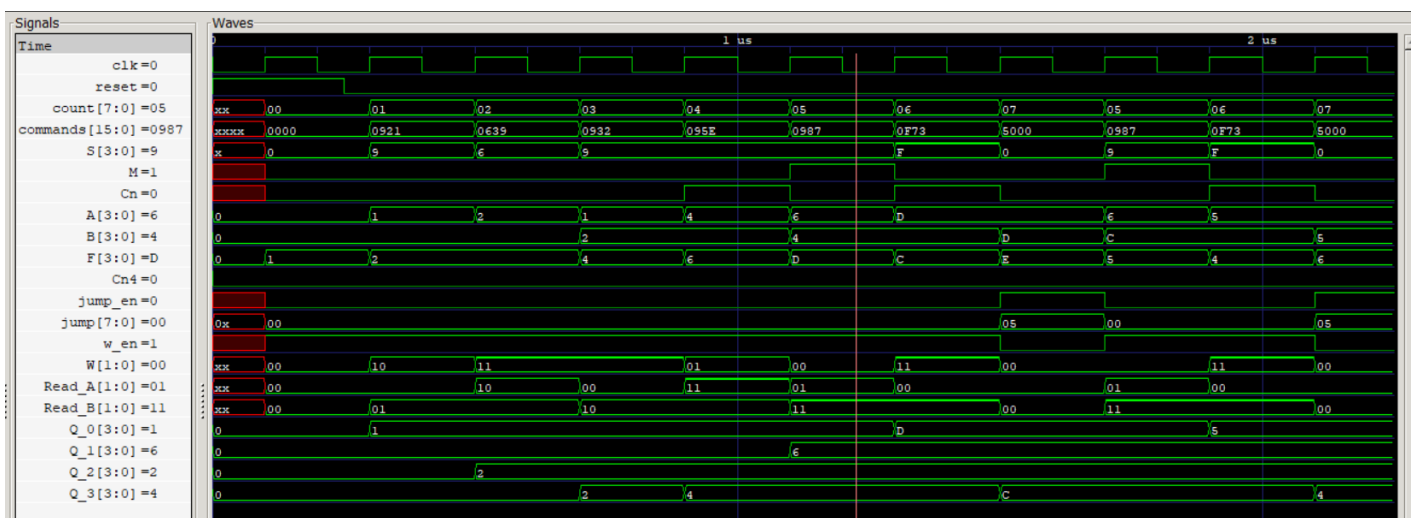


Figure 6: Waveform of the simulation (GTKWave)

7. Conclusion:

The custom 4-bit Harvard architecture microprocessor was successfully designed, simulated at the Register-Transfer Level (RTL), and verified using Icarus Verilog and GTKWave. By utilizing a hybrid Opcode/Address decoding scheme, the Instruction Set Architecture (ISA) achieved high instruction density, effectively minimizing control logic complexity. The simulation waveforms confirm that the datapath executes both 74181-based arithmetic and logic operations, and absolute branching instructions accurately within a single-cycle execution model.

Key architectural learnings:

Throughout the design and verification process, several core concepts of digital logic and computer architecture were practically reinforced:

- **Synchronous vs. Combinational Delays:** Waveform analysis confirmed the physical reality of synchronous hardware, specifically observing the mandatory one-clock-cycle delay required to latch a new target address (JUMP operation) into the Program Counter during a branch operation.
- **Managing "Don't Care" States:** Because the ALU is purely combinational, it continuously calculates outputs even during branch cycles. This project demonstrated how to securely manage these "Don't Care" garbage states by strictly tying the Register File's Write Enable (`w_en`) to the inverse of the Jump Enable (`jump_en`) signal which kept the stored values in register intact during "JUMP" operation.
- **Structural vs. Behavioral RTL:** The project successfully integrated behavioral logic (the FSM Control Unit) with strict structural routing (the top-level CPU module buses), proving the viability of the closed-loop datapath.